Introduction to UML (Unified Modeling Language)

Minsoo Ryu

Hanyang University

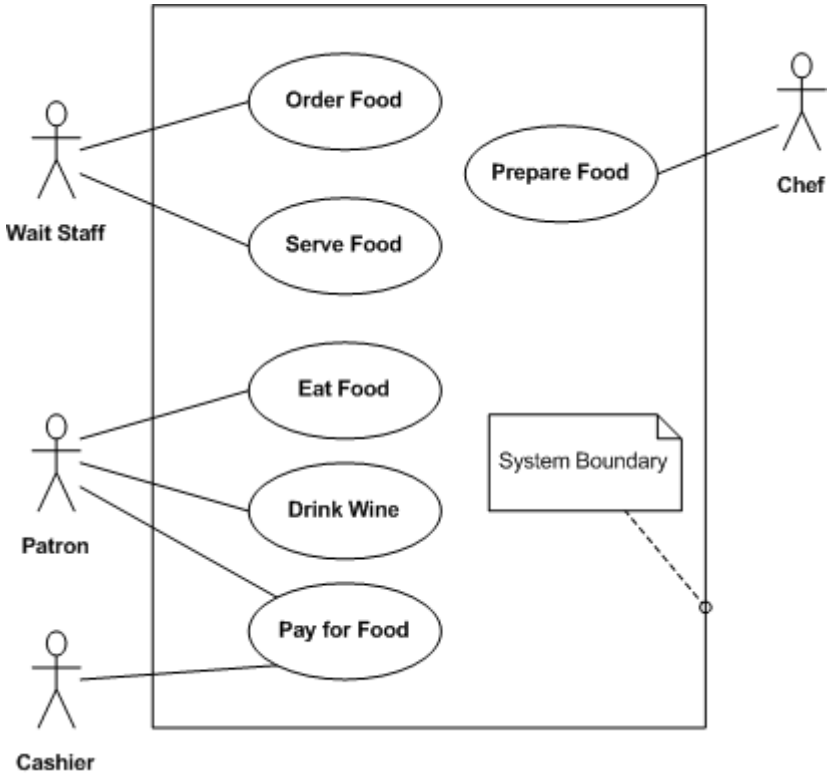
msryu@hanyang.ac.kr

Contents

1. Introduction to UML
2. Structural Diagrams
3. Behavioral Diagrams
   - Use Case Diagram
   - State Machine Diagram
   - Activity Diagram
   - Communication Diagram
   - Sequence Diagram

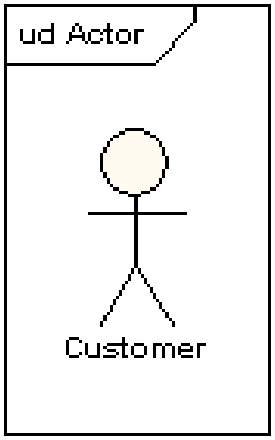
Use Case Diagram

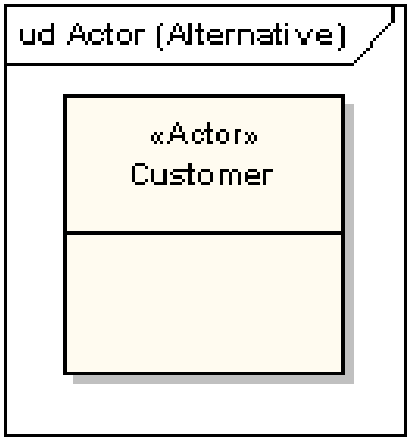
- The use case model captures the requirements of a system
  - Use cases are a means of communicating with users and other stakeholders what the system is intended to do



Actors

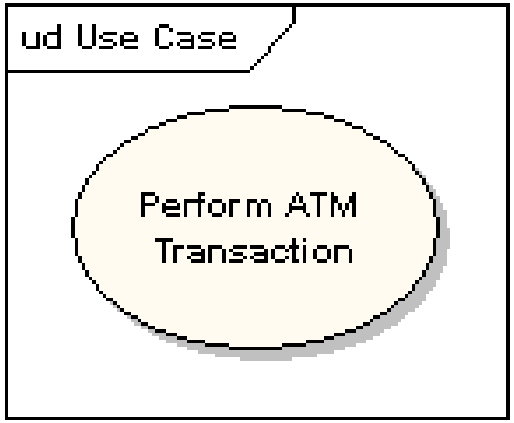
- A use case diagram shows the interaction between the system and entities external to the system
- These external entities are referred to as actors
- Actors represent roles which may include human users, external hardware or other systems





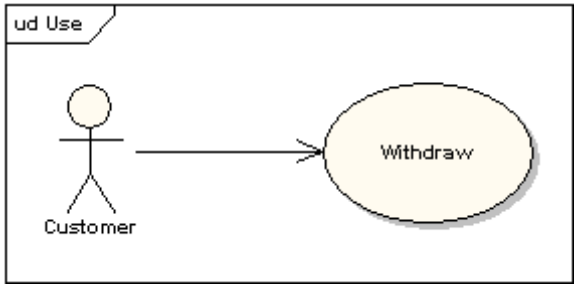
Use Cases

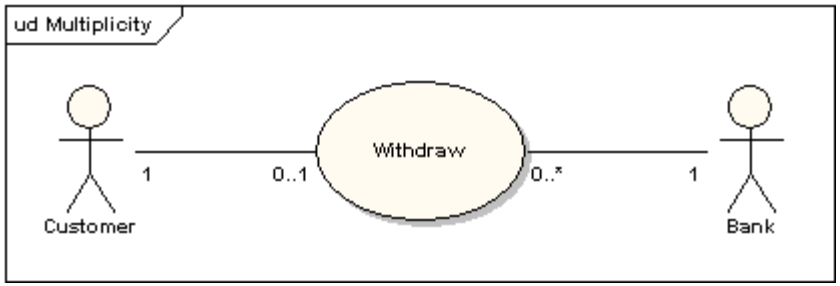
- A use case is a single unit of meaningful work
- It provides a high-level view of behavior observable to someone or something outside the system



Uses Connector

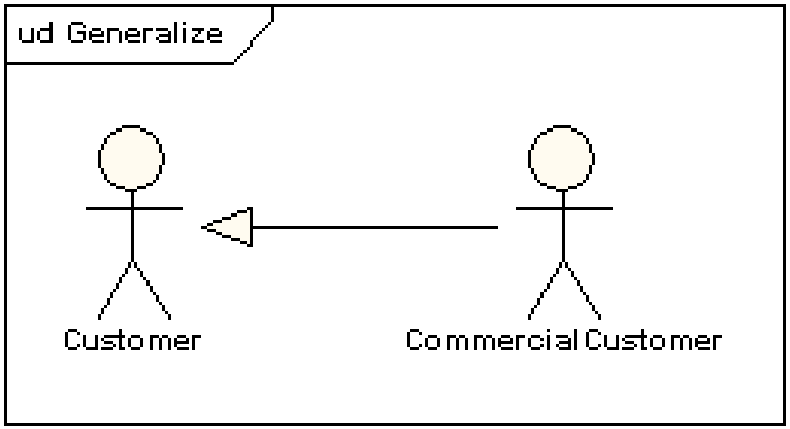
- The notation for using a use case is a connecting line with an optional arrowhead showing the direction of control
- The uses connector can optionally have multiplicity values at each end
  - In the following diagram, a customer may only have one withdrawal session at a time, but a bank may have any number of customers making withdrawals concurrently





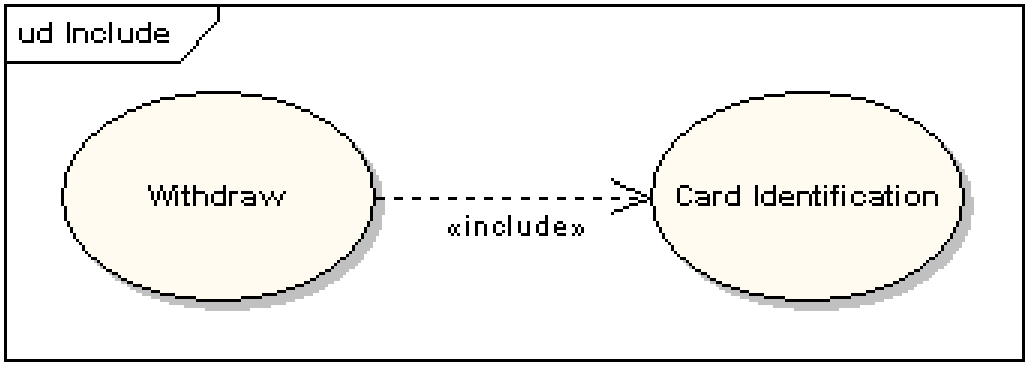
Generalization

- Actors can generalize other actors as detailed in the following diagram



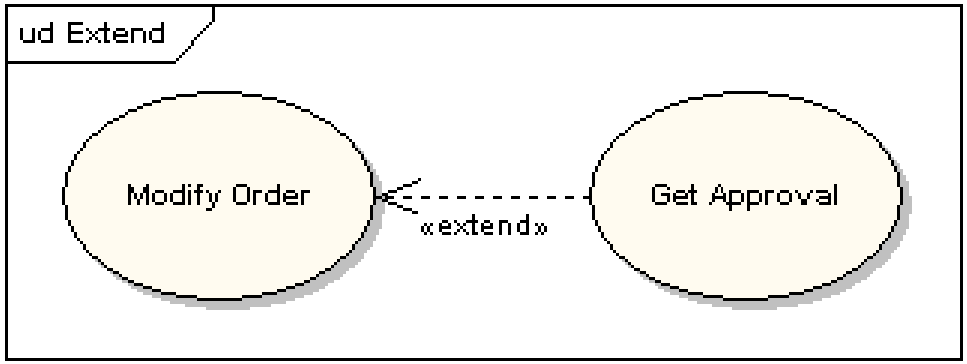
Include Relationship

- Use cases may **contain the functionality of another use case as part of their normal processing**
  - In general it is assumed that any included use case will be called every time the basic path is run



Extend Relationship

- One use case may be used to extend the behavior of another; this is typically used in exceptional circumstances
  - For example, if before modifying a particular type of customer order, a user must get approval from some higher authority, then the <Get Approval> use case may optionally extend the regular <Modify Order> use case

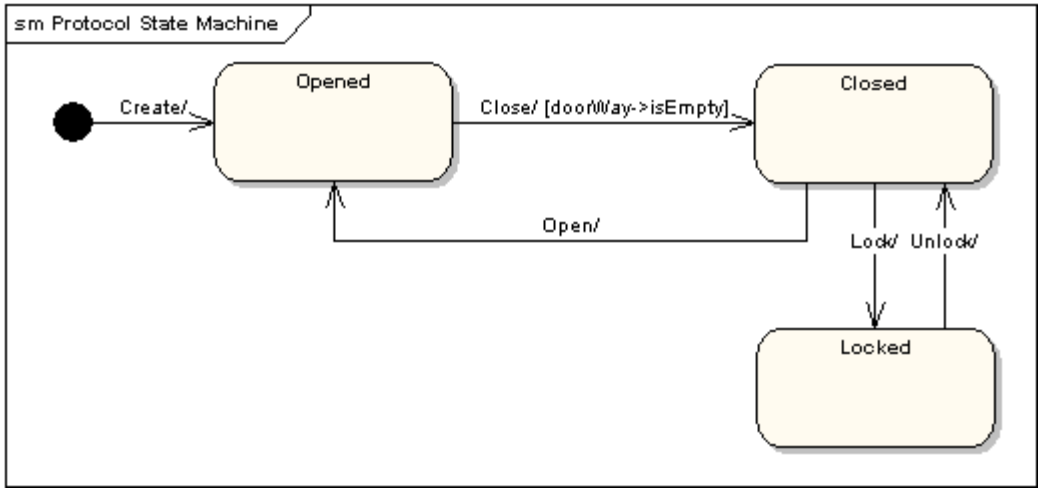


Contents

1. Introduction to UML
2. Structural Diagrams
3. Behavioral Diagrams
   - Use Case Diagram
   - State Machine Diagram
   - Activity Diagram
   - Communication Diagram
   - Sequence Diagram

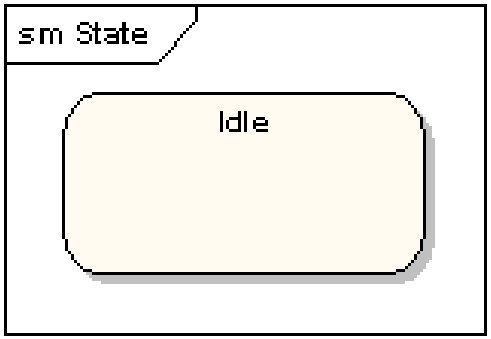
State Machine Diagram

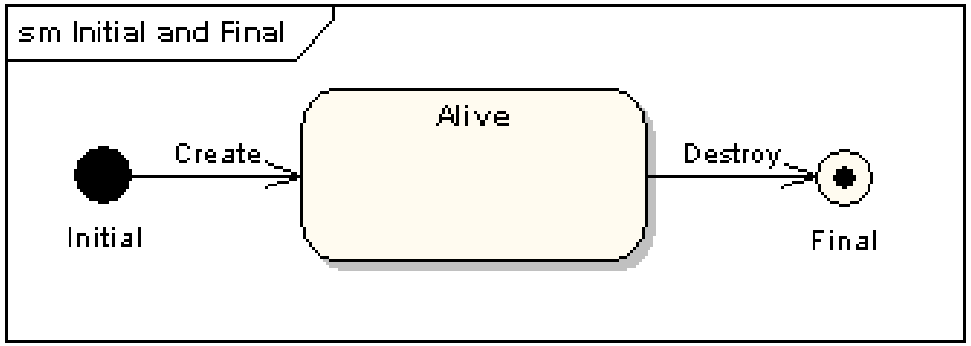
- A state machine diagram models the behavior of a single object, specifying the sequence of events and responses that an object goes through during its lifetime



States

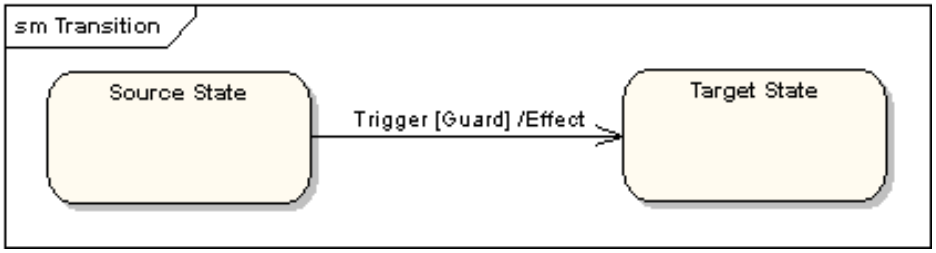
- A state is denoted by a round-cornered rectangle with the name of the state written inside it
- The initial state is denoted by a filled black circle and may be labeled with a name
- The final state is denoted by a circle with a dot inside and may also be labeled with a name





Transitions

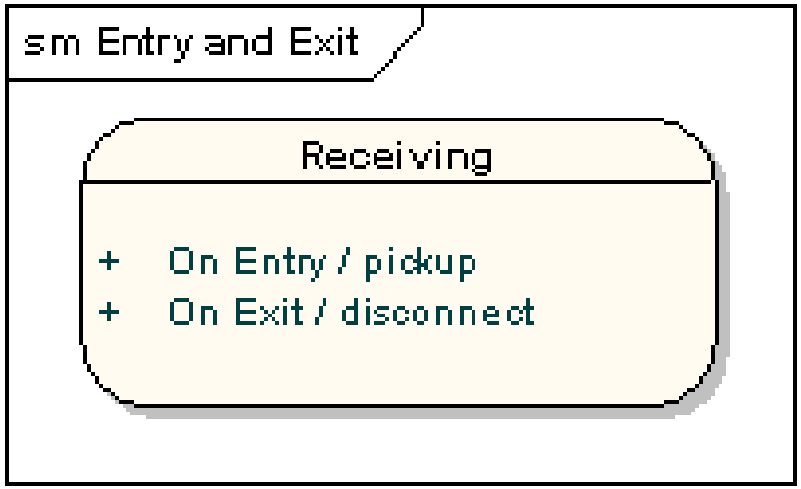
- Transitions from one state to the next are denoted by lines with arrowheads



  - "Trigger" is the cause of the transition, which could be a signal, an event, a change in some condition, or the passage of time
  - "Guard" is a condition which must be true in order for the trigger to cause the transition
  - "Effect" is an action which will be invoked directly on the object that owns the state machine as a result of the transition

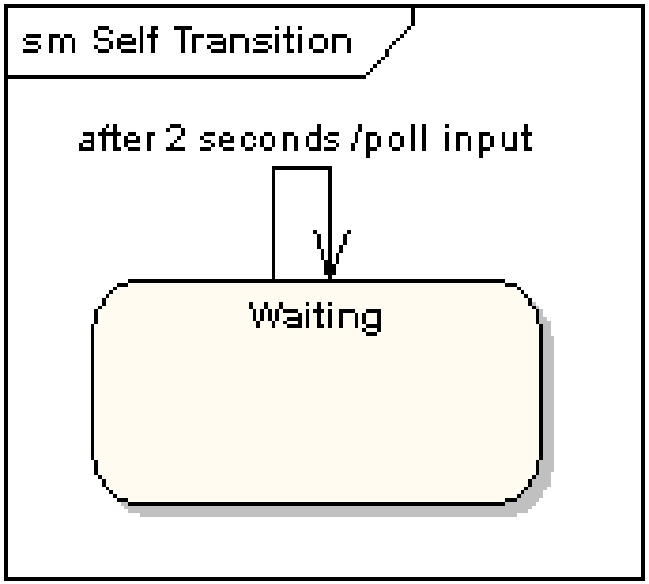
(State) Actions

- On transition
  - An effect can be associated with the transition
- On entry
  - If the target state had many transitions arriving at it, and each transition had the same effect associated with it, it would be better to associate the effect with the target state rather than the transitions
- On exit
  - This can be done by defining an entry action for the state



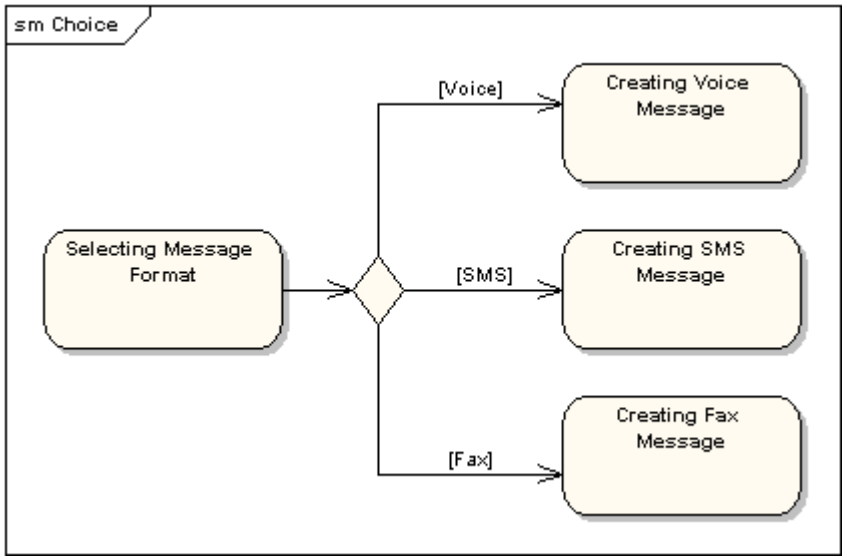
Self Transitions

- A state can have a transition that returns to itself, as in the following diagram
- This is most useful when an effect is associated with the transition



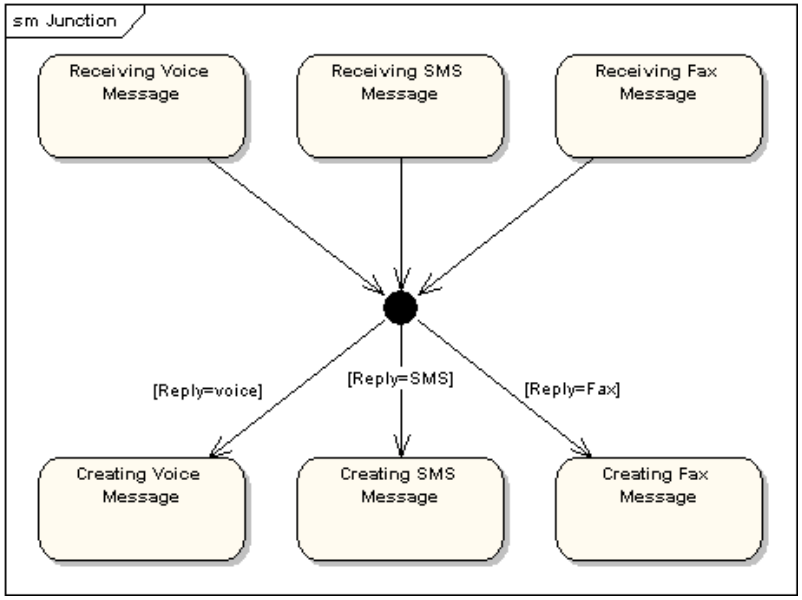
Choice Pseudo-State

- A choice pseudo-state is shown as a diamond with one transition arriving and two or more transitions leaving
  - The following diagram shows that whichever state is arrived at, after the choice pseudo-state, is dependent on the message format selected during execution of the previous state



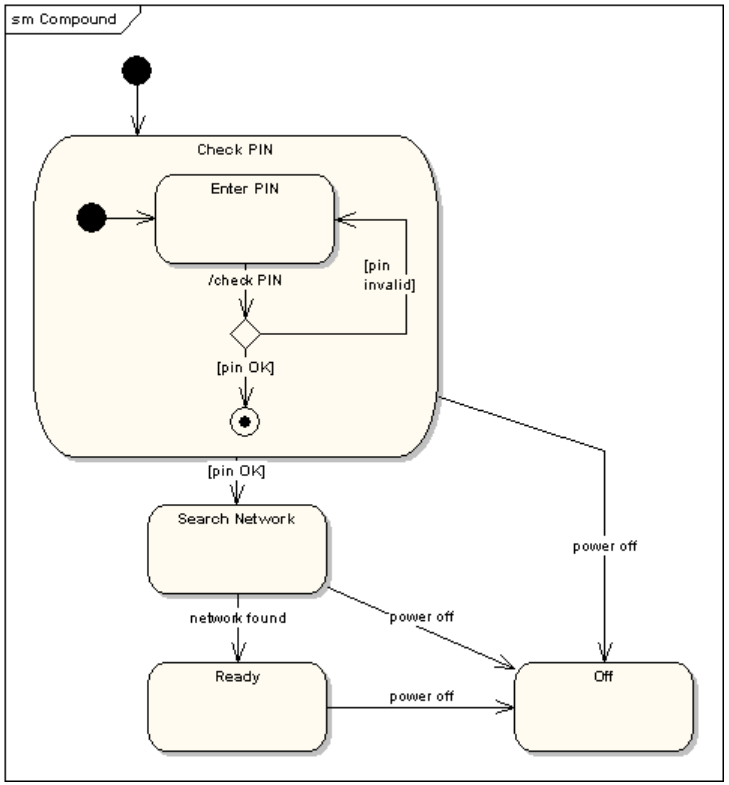
Junction Pseudo-State

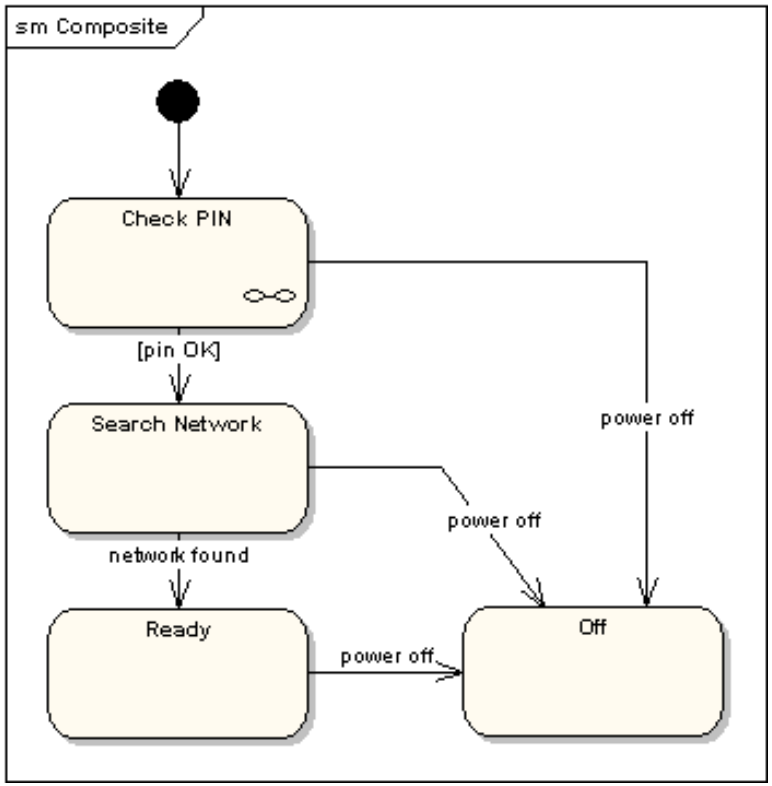
- Junction pseudo-states are used to chain together multiple transitions
  - A single junction can have one or more incoming, and one or more outgoing, transitions; a guard can be applied to each transition (choice only allows one incoming transition)
  - No additional semantics



Compound States

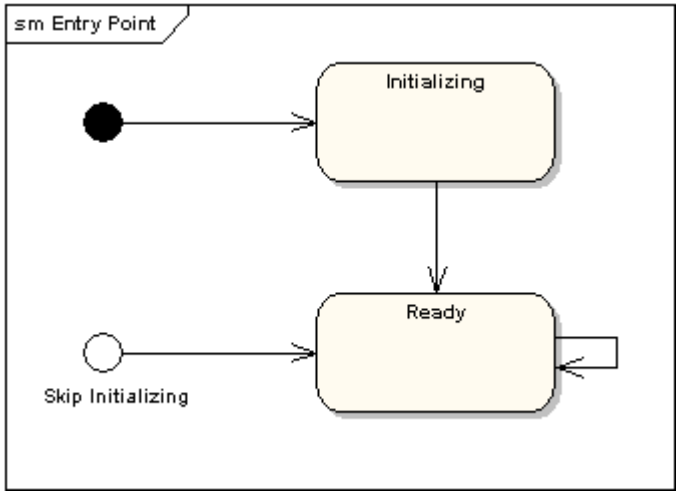
- A state machine diagram may include sub-machine diagrams, as in the example below

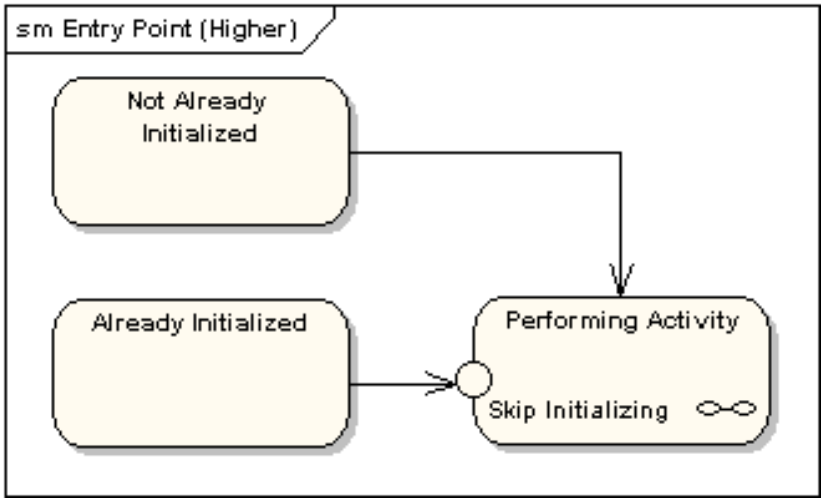




Entry Point

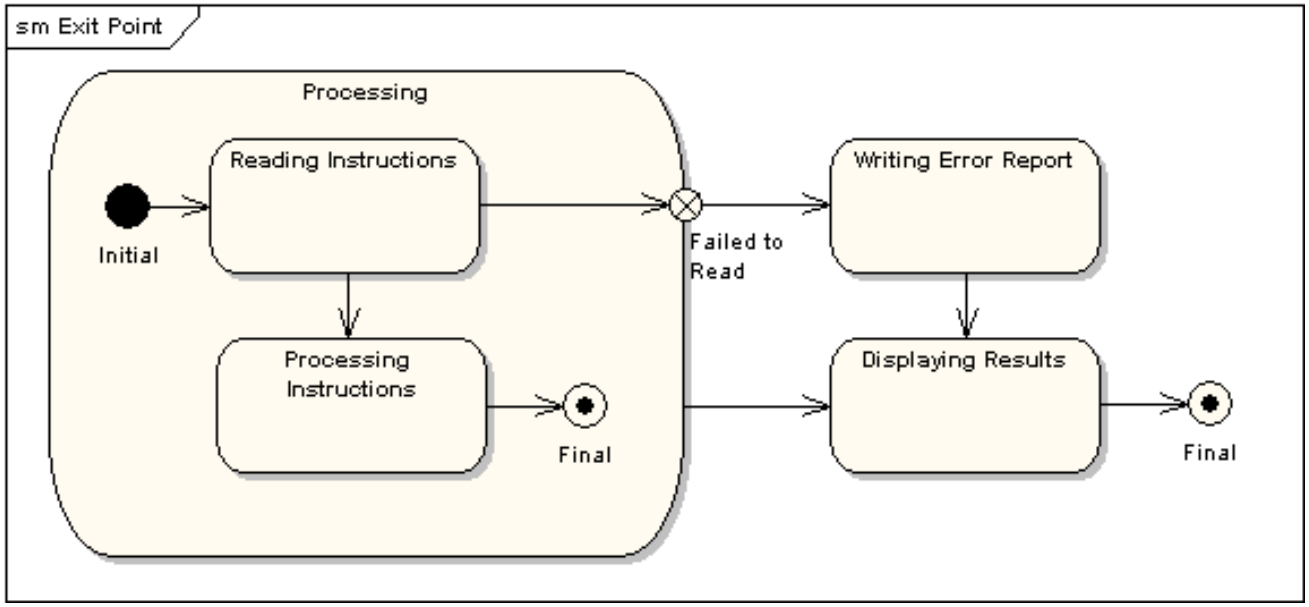
- Entry point is used when you won't want to enter a sub-machine at the normal initial state
  - For example, in the following sub-machine it would be normal to begin in the "Initializing" state, but if for some reason it wasn't necessary to perform the initialization, it would be possible to begin in the "Ready" state by transitioning to the named entry point





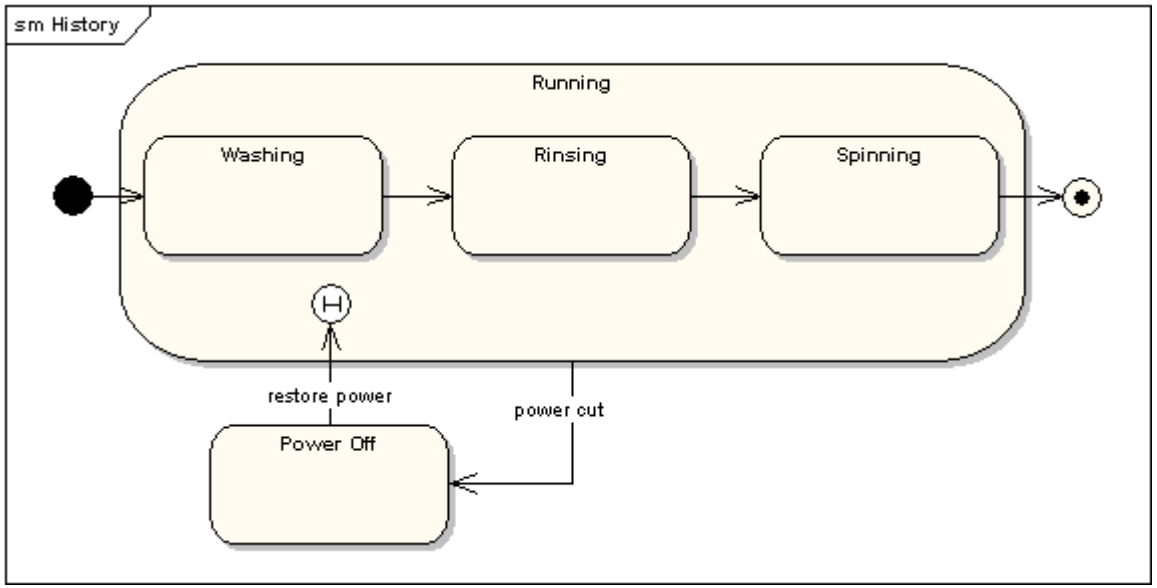
Exit Point

- In a similar manner to entry points, it is possible to have named alternative exit points
  - The following diagram gives an example where the state executed after the main processing state depends on which route is used to transition out of the state



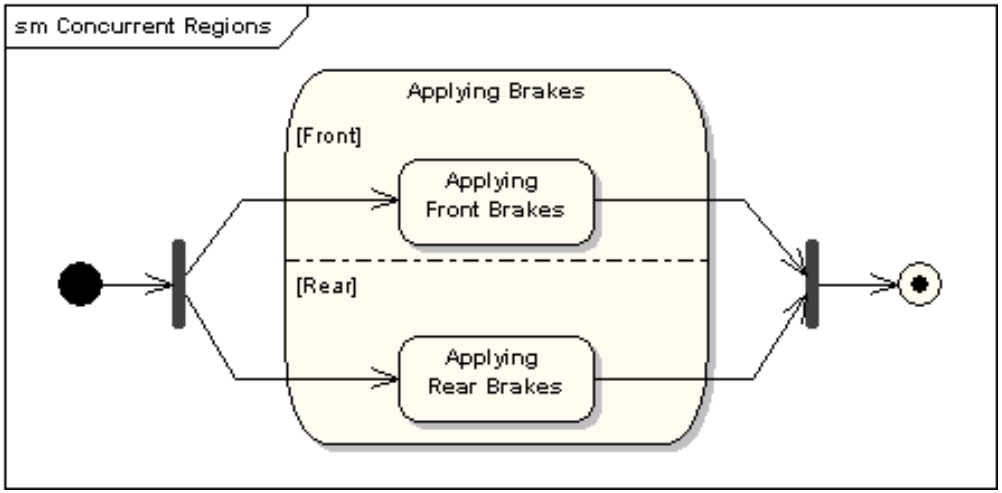
History States

- A history state is used to remember the previous state of a state machine when it was interrupted
  - The following diagram illustrates the use of history states
  - The example is a state machine belonging to a washing machine



Concurrent Regions

- A state may be divided into regions containing sub-states that exist and execute concurrently
  - The example below shows that within the state "Applying Brakes", the front and rear brakes will be operating simultaneously and independently
  - Notice the use of fork and join pseudo-states to synchronize the concurrent threads

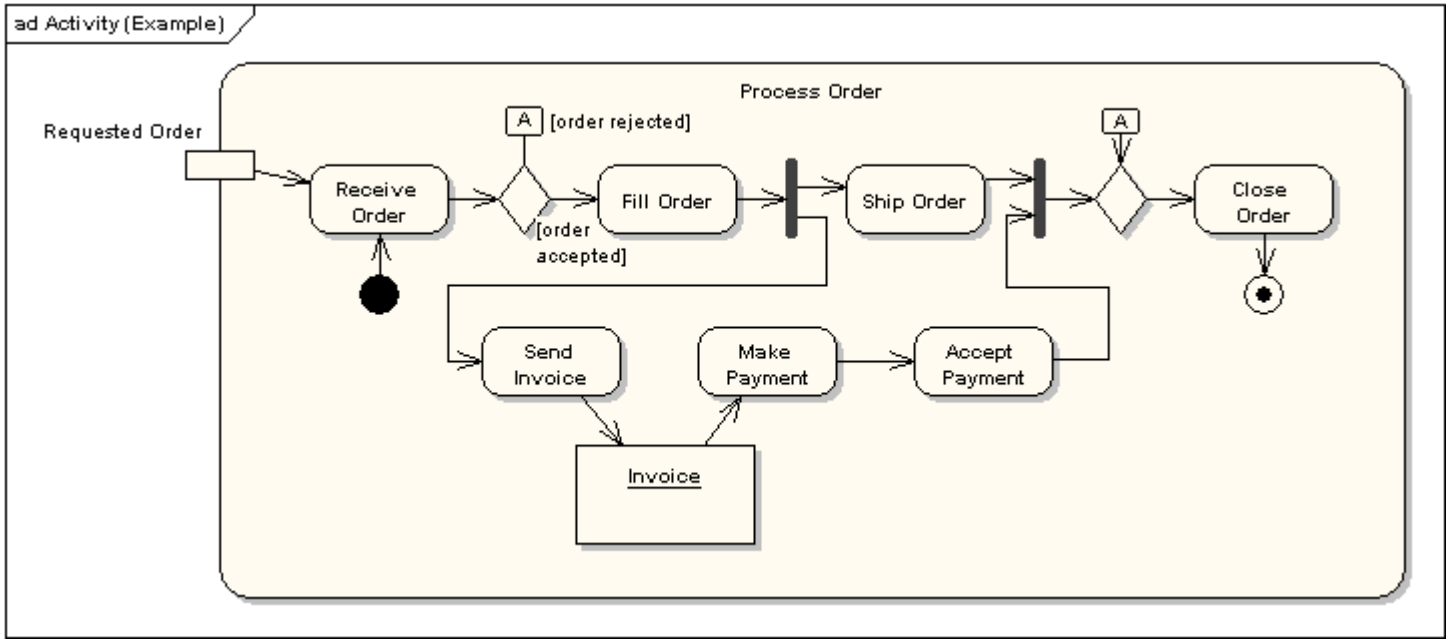


Contents

1. Introduction to UML
2. Structural Diagrams
3. Behavioral Diagrams
   - Use Case Diagram
   - State Machine Diagram
   - Activity Diagram
   - Communication Diagram
   - Sequence Diagram

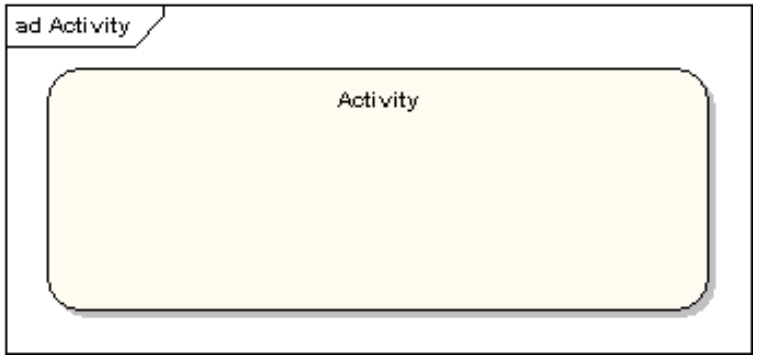
Activity Diagram

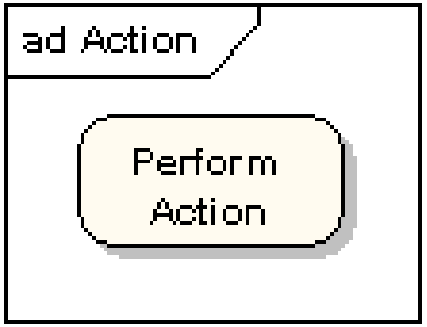
- An activity diagram is used to display the sequence of activities
  - They may be used to detail situations where parallel processing may occur in the execution of some activities



Activities and Actions

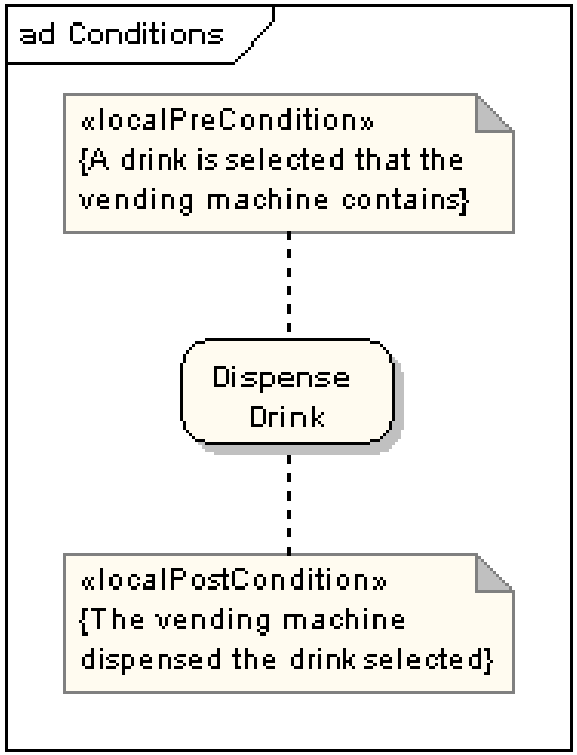
- An activity is the specification of a parameterized sequence of behavior
  - An activity is shown as a round-cornered rectangle enclosing all the actions, control flows and other elements that make up the activity
- An action represents a single step within an activity
  - Actions are denoted by round-cornered rectangles





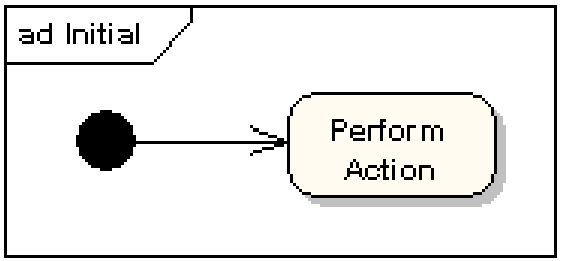
Action Constraints

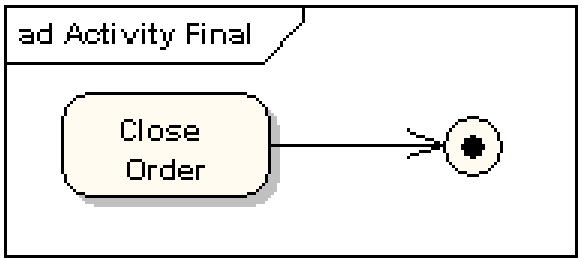
- Constraints can be attached to an action
- The following diagram shows an action with local pre- and post-conditions



Initial Node and Final Node

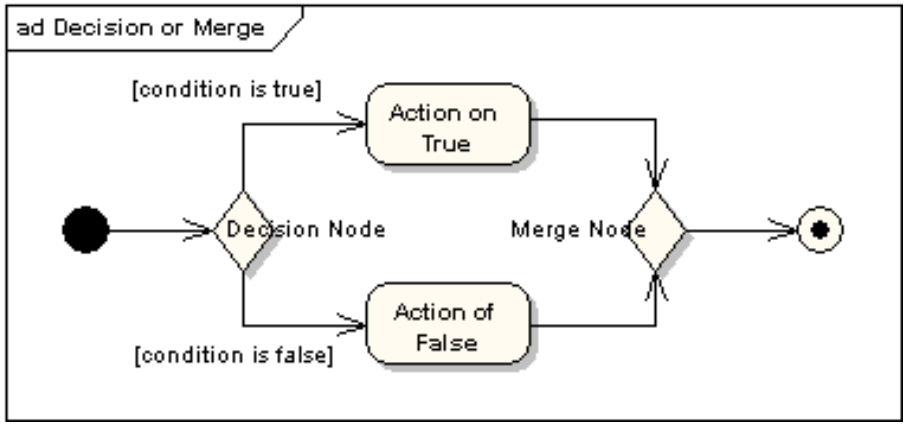
- An initial or start node is depicted by a large black spot
- There are two types of final node: activity and flow final nodes
  - The activity final node indicates that the activity has been completed
  - The flow final node indicates that a single flow within an activity is complete





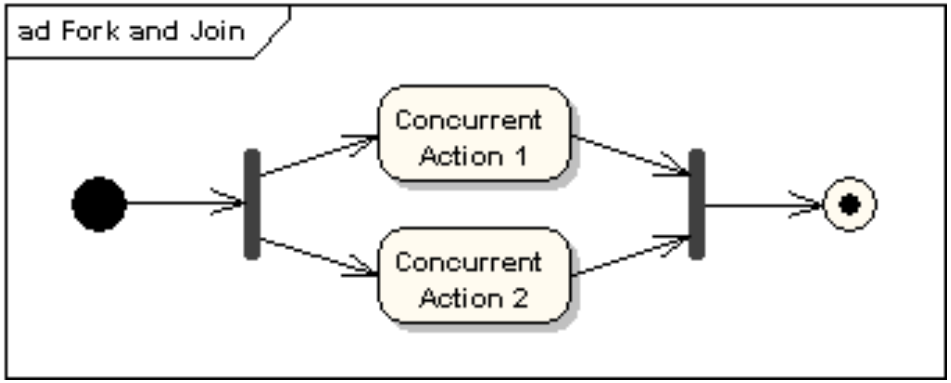
Decision and Merge Nodes

- Decision nodes and merge nodes have the same notation: a diamond shape
  - They can both be named
  - The control flows coming away from a decision node will have guard conditions which will allow control to flow if the guard condition is met
  - The following diagram shows use of a decision node and a merge node



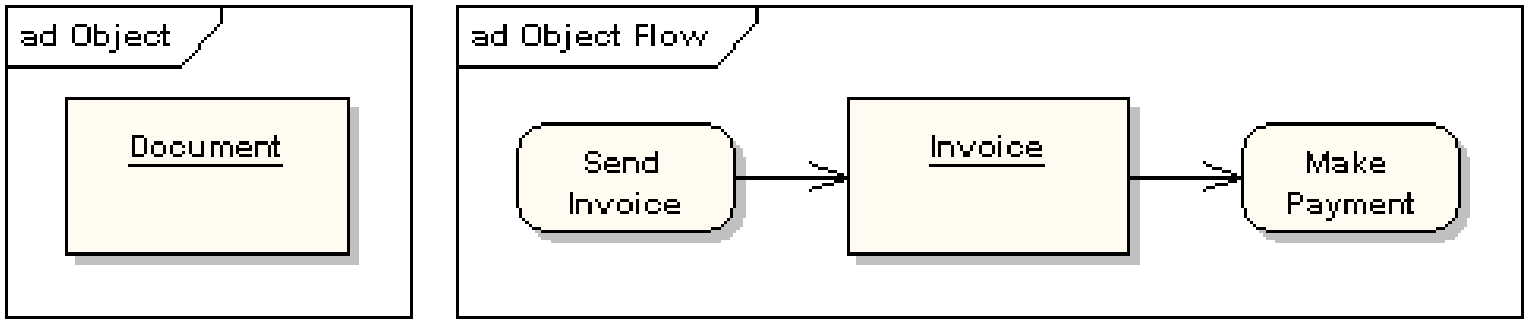
Fork and Join Nodes

- Forks and joins have the same notation: either a horizontal or vertical bar
  - They indicate the start and end of concurrent threads of control
  - A join is different from a merge in that the join synchronizes two inflows and produces a single outflow
  - The outflow from a join cannot execute until all inflows have been received



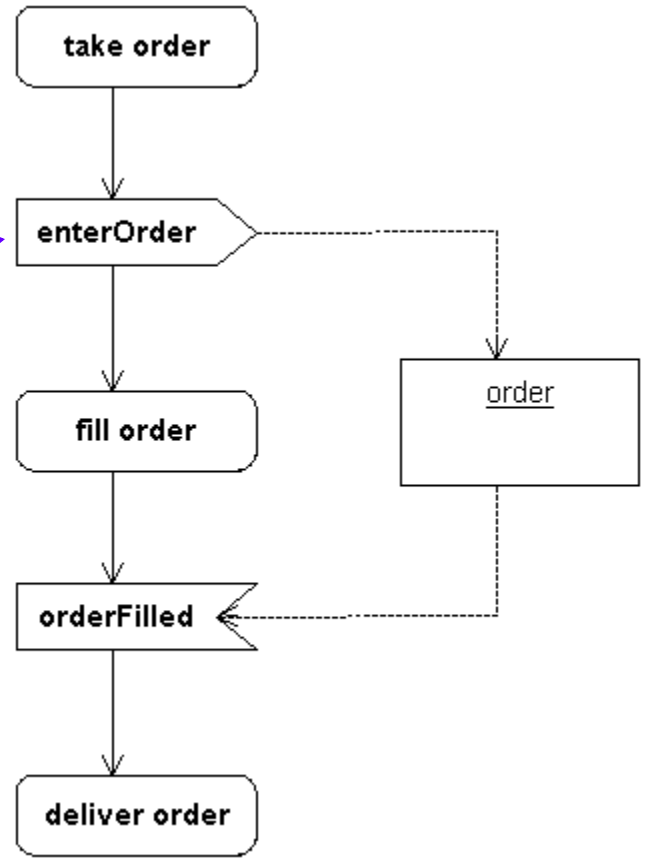
Objects and Object Flows

- An object flow is a path along which objects or data can pass
- An object flow is shown as a connector with an arrowhead denoting the direction the object is being passed



Signals

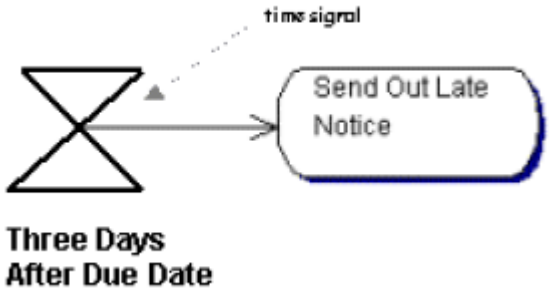
- **When an activity sends or receives a message, that activity is called a signal**



- **Input signal**
  - message receiving activity
- **Output signal**
  - message sending activity

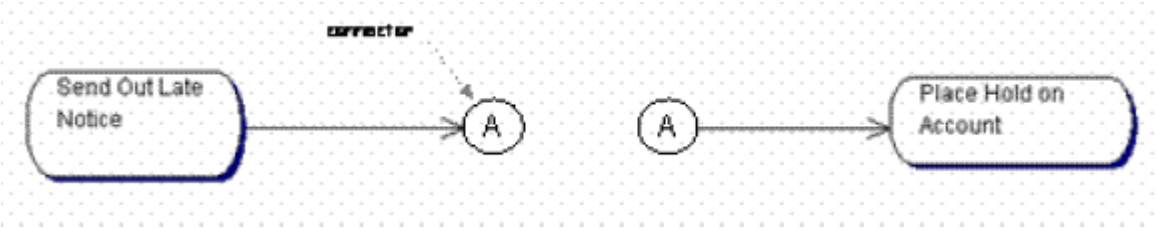
Time Signal

- The time signal is a special kind of a signal and denoted by a hour glass
  - A time signal is generated when a certain amount of time has passed



Connector

- "Off-page" connectors are used to allow to span multiple pages
  - There are two instances, a target of a flow and a source of a flow
  - Both instances have the same name
  - The connector indicates that a flow moves from one activity to another



Contents

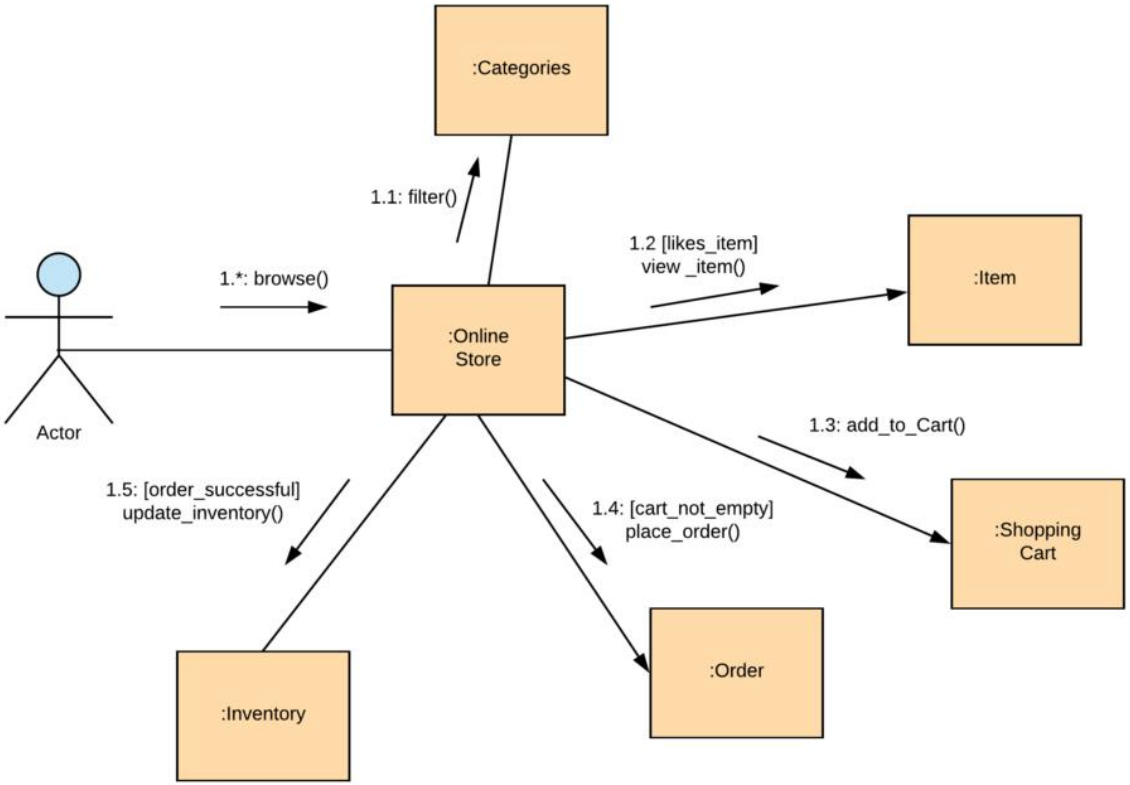
1. Introduction to UML
2. Structural Diagrams
3. Behavioral Diagrams
   - Use Case Diagram
   - State Machine Diagram
   - Activity Diagram
   - Communication Diagram
   - Sequence Diagram

Communication Diagram

- A communication diagram, formerly called a collaboration diagram, is an interaction diagram that shows similar information to sequence diagrams but its primary focus is on object interactions

- On communication diagrams, objects are shown with association connectors between them
  - Messages are added to the associations and show as short arrows pointing in the direction of the message flow
  - The sequence of messages is shown through a numbering scheme

Communication Diagram



Contents

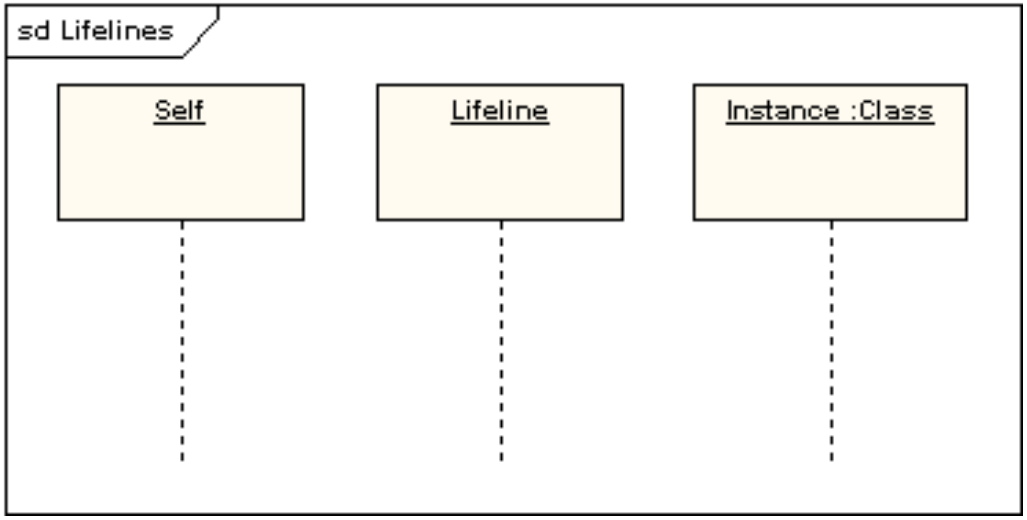
1. Introduction to UML
2. Structural Diagrams
3. Behavioral Diagrams
   - Use Case Diagram
   - State Machine Diagram
   - Activity Diagram
   - Communication Diagram
   - Sequence Diagram

Sequence Diagram

- A sequence diagram is a form of interaction diagram which shows objects as lifelines running down the page, with their interactions over time represented as messages drawn as arrows from the source lifeline to the target lifeline
  - Sequence diagrams are good at showing which objects communicate with which other objects; and what messages trigger those communications
  - Sequence diagrams are not intended for showing complex procedural logic

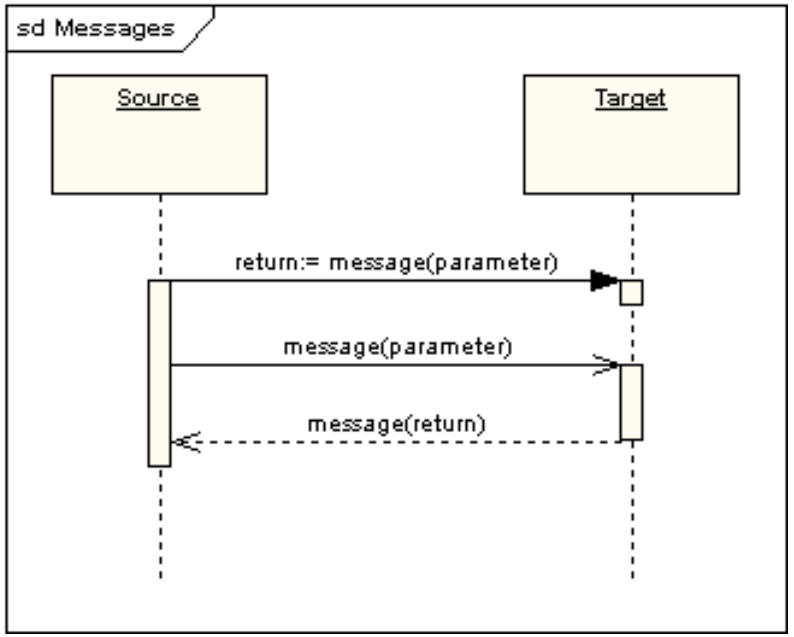
Lifelines

- A lifeline represents an individual participant in a sequence diagram
  - A lifeline will usually have a rectangle containing its object name



Messages

- Messages are displayed as arrows
  - A synchronous message (denoted by the solid arrowhead) operation does not allow the source object to continue its next step until the target object complete the flow of control
  - An asynchronous message (denoted by line arrowhead) operation allows the source object to immediately continues with the next step

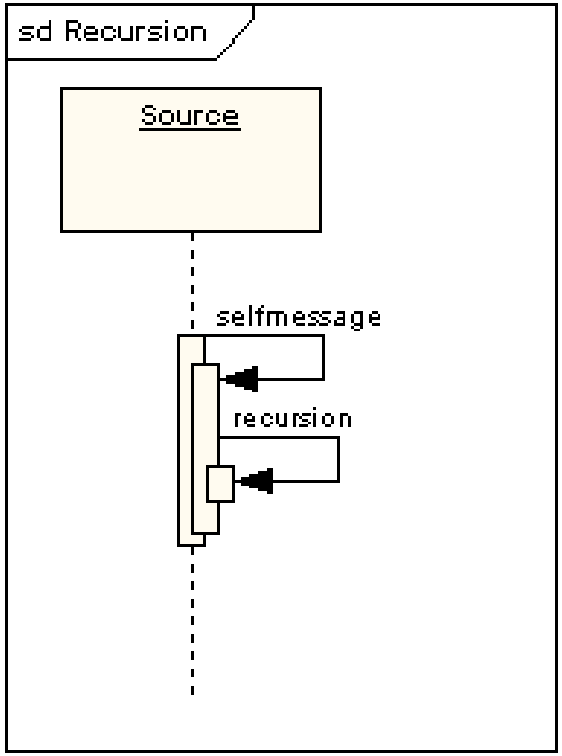


Execution Occurrence

- A thin rectangle running down the lifeline denotes the execution occurrence, or activation of a focus of control
  - In the previous diagram, there are three execution occurrences
  - The first is the source object sending two messages and receiving two replies; the second is the target object receiving a synchronous message and returning a reply; and the third is the target object receiving an asynchronous message and returning a reply

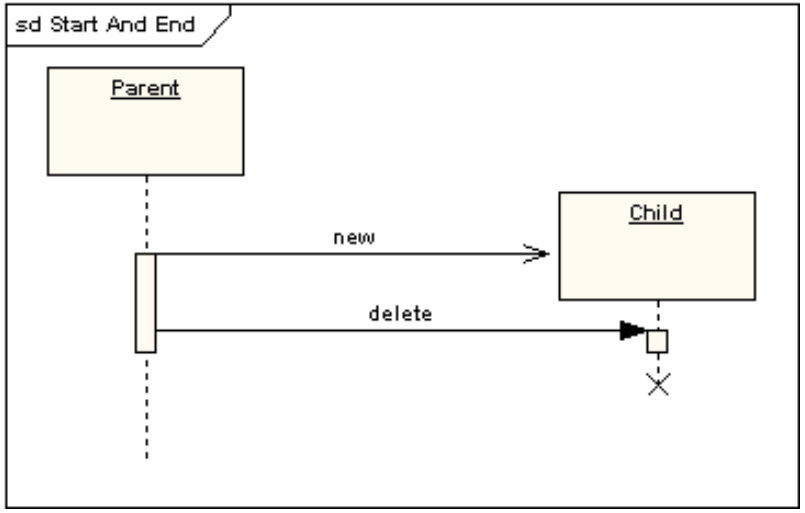
Self Message

- A self message can represent a recursive call of an operation, or one method calling another method belonging to the same object



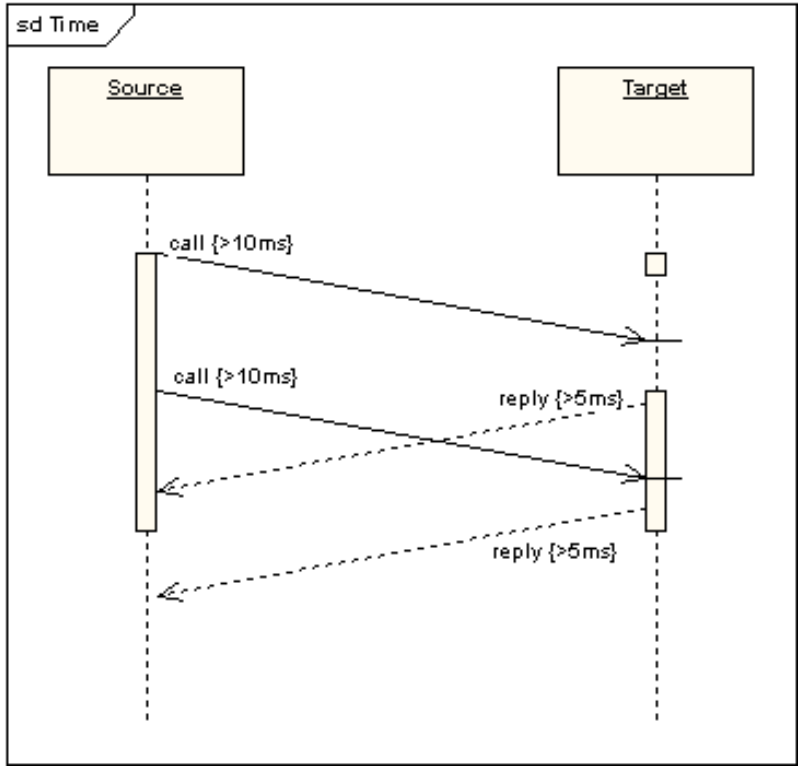
Lifeline Start and End

- A lifeline may be created or destroyed during the timescale represented by a sequence diagram
  - In the latter case, the lifeline is terminated by a stop symbol, represented as a cross
  - In the former case, the symbol at the head of the lifeline is shown at a lower level down the page than the symbol of the object that caused the creation



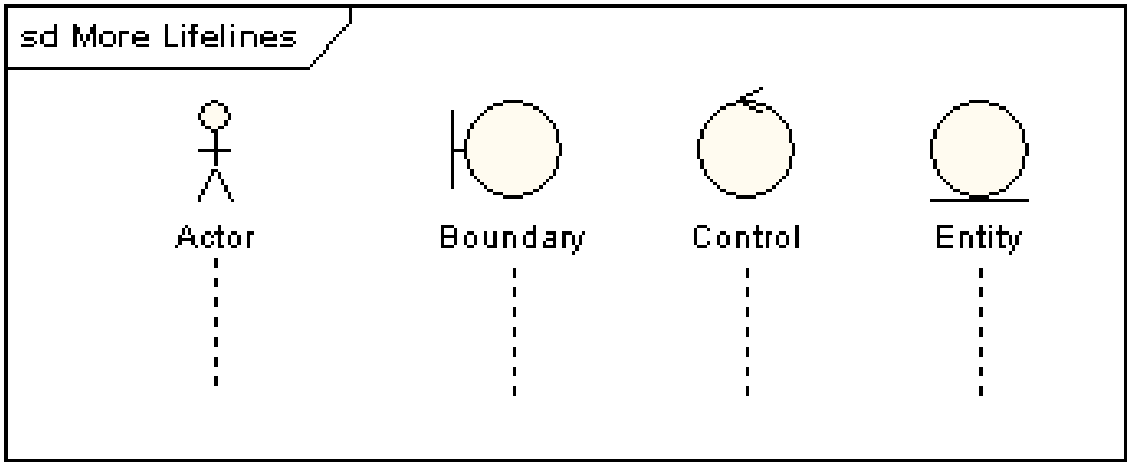
Duration and Time Constraints

- By default, a message is shown as a horizontal line. Since the lifeline represents the passage of time down the screen, when modelling a real-time system, or even a time-bound business process, it can be important to consider the length of time it takes to perform actions
  - By setting a duration constraint for a message, the message will be shown as a sloping line



Sequence Diagrams for Use Cases

- Sometimes a sequence diagram will have a lifeline with an actor element symbol at its head
  - This will usually be the case if the sequence diagram is owned by a use case
  - Boundary, control and entity elements from robustness diagrams can also own lifelines




Q & A

다이어그램에 대하여 …

- **UML** 대신 자신만의 기호**(notation)**를 사용하는 경우가 많다
  - **UML**을 모르거나 또는 객체지향을 사용하지 않는 상황이 빈번하기 때문이다
- 그러나 다이어그램은 결국 도형**(shape)**과 선**(line)**의 집합이다
  - 사람마다 도형과 선의 의미가 다를 뿐이다
- 따라서 “기호 일관성**(consistency of notation)**”을 지켜야 한다
  - 도형별(사각형, 원, …) 의미, 연결선별(실선, 점선, 화살선, …) 의미
  - **UML**의 **relationship**에서 보듯이 연결선은 다양한 의미를 가질 수 있다
- 개념적 측면에서는 “추상수준 일관성**(consistency of abstraction level)**”을 지켜야 한다
  - 예: 메신저에서 문자 대화, 음성 대화, 영상 대화, 쇼핑하기를 모두 파란 사각형으로 표현 (쇼핑하기는 분류 기준과 추상화 수준이 다르다)